GoTrain

An autonomous AI scientist that trains geospatial models on its own

AUB ACM × RASID Hackathon

By Team 8: Charbel, Marwa and Rabih

Technical Report for the Jury

95.7%	**97.9%**	**0.951**	**0.844**
EuroSAT accuracy	PV4GER accuracy	PV4GER-seg mIoU	NZ-cattle mIoU

Final test scores, each test split opened exactly once. About 64 GPU-minutes on one Tesla T4.

Contents

1) Executive Summary **1**

2) The Challenge and Our Ground Rules **1**

3) System Architecture **2**

4) Data Pipeline: Getting the Details Right **3**

5) Models, Losses and Optimisation **4**

6) The Agent: How GoTrain Decides **6**

7) How We Achieved the Accuracy **7**

8) Final Test Results **10**

9) What Makes GoTrain Innovative **10**

10) Evaluation Integrity and Engineering **12**

11) Honest Limitations **13**

12) What We Would Build Next at RASID **14**

13) Appendix: Reproduce and Verify **14**

1 Executive Summary

Earth-observation models are still built by hand: an expert picks a backbone, designs a training recipe and tunes it, again for every new sensor and task. **GoTrain** replaces that loop with an agent. Given a GEO-Bench dataset and a GPU budget, a Strands agent driven by Claude Sonnet on Amazon Bedrock inspects the data, states a hypothesis, trains a model on the SageMaker GPU, measures it on the validation split, and decides what to try next. Every decision is logged, and the test split is opened exactly once, at the end, by a separate locked command.

Table 1: Final test results. Selection used validation data only; each test split was evaluated once.

Task	Type	Test images	Metric	Final test	Best validation
`m-eurosat`	10-class classification	1,000	accuracy	**95.70%**	97.20%
`m-pv4ger`	binary classification	999	accuracy	**97.90%**	97.50%
`m-pv4ger-seg`	binary segmentation	403	mean IoU	**0.9514**	0.9526
`m-nz-cattle`	binary segmentation	65	mean IoU	**0.8439**	0.8476

What we want the jury to remember:

1. **Strong scores from a disciplined process.** A manual CNN reached 75% on EuroSAT; the agent's pipeline reached 97% validation and 95.7% test. Validation and test agree closely on three of four tasks, which indicates we did not overfit to validation.
2. **Every gain is explained by a measured hypothesis.** For example, the agent noticed that cattle cover only 2.2% of pixels and raised the input resolution (+0.014 mIoU); it noticed 36 missed solar panels and turned on class balancing (+1.0 point accuracy).
3. **An AI scientist, not a tuner.** GoTrain diagnoses where a model fails (object detection rate by size, confused classes, worst images), proposes a targeted fix, keeps it only if validation improves, and **remembers** the outcome across datasets.
4. **Production-minded.** Immutable model versions with SHA-256 checksums, a model registry with verified restores, a ChatGPT-style interface, a results dashboard, 34 automated tests and an MCP server so that GoPilot can call the same tools.

2 The Challenge and Our Ground Rules

2.1) The four problems we chose

The brief asked for two classification and two segmentation problems from GEO-Bench. We chose them to cover different sensors and difficulty levels.

2.2) The one rule we enforced in code, not in discipline

Table 2: Selected GEO-Bench tasks (split sizes from the brief; class balance measured on a seeded training sample).

Task	What is predicted	Train / val / test	Input	Imbalance
`m-eurosat`	land cover, 10 classes	2000 / 1000 / 1000	Sentinel-2, 13 bands	roughly balanced
`m-pv4ger`	solar panel present or not	11814 / 999 / 999	aerial RGB, 0.1 m	15.6% positive
`m-pv4ger-seg`	solar panel pixels	3000 / 403 / 403	aerial RGB, 0.1 m	19.2% foreground
`m-nz-cattle`	cattle pixels	524 / 66 / 65	aerial RGB, 0.1 m	**2.2% foreground**

Definition 2.1 (Protocol). The *train* split fits weights. The *valid* split is the only signal used to choose recipes, epochs and checkpoints. The *test* split is opened once per task, after the selected model has been frozen, and never influences any decision.

We did not rely on good intentions. The development data loader refuses any split other than `train` and `valid`; the agent and the MCP server have no test tool at all; and `gotrain finalize` freezes the best validation run, creates an exclusive marker file *before* opening the first test sample, and refuses to run twice (Section 10).

3 System Architecture

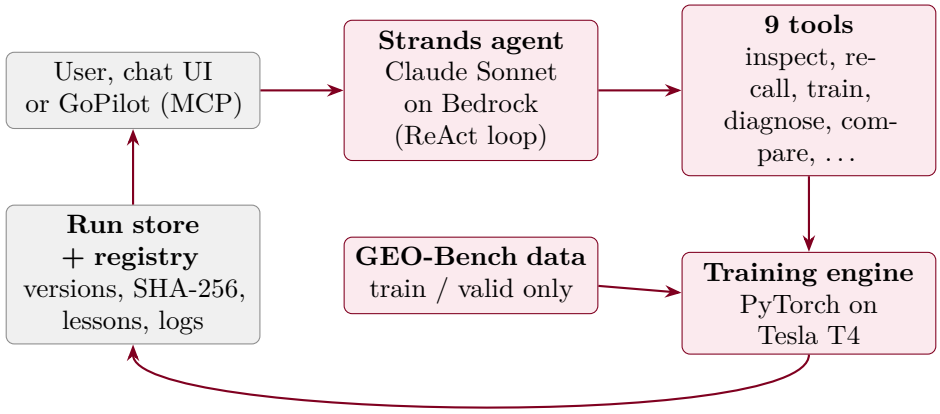


Figure 1: GoTrain at a glance. The agent reasons; the tools act; every result lands in an immutable, checksummed run store.

3.1) Components

- **Agent.** One Strands agent with a system prompt that encodes the protocol, sensible starting

recipes and the scientist workflow (Section 6). It runs a ReAct loop: reason, call a tool, read the measured result, reason again.

- **Tools.** `inspect_dataset`, `recall_lessons`, `run_experiment`, `diagnose_errors`, `record_lesson`, `compare_experiments`, `publish_best_model`, `publish_dataset`, `verify_model_download`. None of them can read the test split.
- **Training engine.** A single PyTorch pipeline for both problem types: data loading and caching, model construction, training with early stopping and a wall-clock budget, and validation metrics from a dataset-wide confusion matrix.
- **Storage.** Every run is an immutable directory named by timestamp and UUID containing its configuration, hypothesis, normalisation statistics, per-epoch history, event log, best checkpoint and SHA-256 hash. A registry publishes and restores versions with checksummed manifests.
- **Interfaces.** A command line, a chat web app (`gotrain ui`), an offline results dashboard, and a stdio MCP server exposing seven tools to GoPilot.

3.2) Stack

Python 3.12, PyTorch 2.7.1 and torchvision 0.22.1 (CUDA 12.6), geobench 1.1.0, strands-agents 1.63.0, Claude Sonnet (`eu.anthropic.claude-sonnet-5-5`, `eu-central-1`), one NVIDIA Tesla T4 on Amazon SageMaker. Peak GPU memory of any run was 1.2 GB, so the T4 was never the bottleneck.

4 Data Pipeline: Getting the Details Right

High accuracy starts with not corrupting the data. Each item below fixed or prevented a real failure mode.

4.1) Normalisation from training data only

Definition 4.1 (Train-only normalisation). For each input channel c, the mean μ_c and standard deviation σ_c are estimated from a seeded random sample of 128 *training* images at the training resolution, and every input is transformed as $\tilde{x}_c = (x_c - \mu_c)/\sigma_c$. The sampled indices are stored with each run, so the statistics are reproducible.

We deliberately did not use benchmark-wide statistics: they can be computed on other splits or a different band selection, which silently leaks information.

4.2) Thirteen spectral bands into an ImageNet backbone

EuroSAT has 13 Sentinel-2 bands, while pretrained ResNets expect 3. Rather than dropping 10 bands or training the first layer from scratch, we adapt the first convolution:

Definition 4.2 (First-layer adaptation). Let $W \in \mathbb{R}^{64\times3\times7\times7}$ be the pretrained first-layer weights. For C input channels the new weights are

$$W'_{:,c,:,:} = \frac{3}{C} \cdot \frac{1}{3} \sum_{j=1}^{3} W_{:,j,:,:}, \qquad c = 1, \ldots, C,$$

so that every band starts with the averaged RGB filters and the expected activation magnitude matches what the pretrained network was trained on.

We also found that EuroSAT's published band-name metadata is unreliable, so channels are always used in their physical stored order and never selected by name.

4.3) Augmentation that keeps masks aligned

Training samples receive random horizontal and vertical flips and a random multiple of 90° rotation (the dihedral group of the square, a natural symmetry of nadir imagery). For segmentation the identical transform is applied to the mask, which is resized with nearest-neighbour interpolation so labels are never blended. A unit test checks image and mask alignment.

4.4) Evaluation at native label resolution

Training may run on resized masks to fit the GPU, but validation and test predictions are upsampled back to the *native* annotation resolution before scoring, so the metric is computed on the original labels.

4.5) Throughput

Resized development images are cached once as memory-mapped float16 tensors (when the payload is below 2 GiB), while normalisation and random augmentation are still applied per sample. With two data-loader workers overlapping I/O and GPU work, a full EuroSAT training run takes about six minutes.

5 Models, Losses and Optimisation

5.1) Architectures

- **Classification:** ImageNet-pretrained ResNet-18 or ResNet-34 with a new linear head (first layer adapted as in Definition 4.2).
- **Segmentation:** a U-Net-style network with the pretrained ResNet as encoder. Four decoder stages upsample and concatenate the encoder skip connections ($512 \to 256 \to 128 \to 64 \to 32$ channels, GroupNorm, ReLU), followed by a 1×1 convolution and bilinear upsampling to the input size.

- A small randomly initialised CNN is kept for smoke tests and CPU checks.

5.2) Loss functions

Definition 5.1 (Class-balanced cross-entropy). With n_k training labels of class k (pixels or images) and $N = \sum_k n_k$,

$$w_k = \mathrm{clip}\left(\frac{\sqrt{N/n_k}}{\frac{1}{K}\sum_j \sqrt{N/n_j}},\ 0.1,\ 10 \right), \qquad \mathcal{L}_{\mathrm{CE}} = -\sum_i w_{y_i} \log p_{i,y_i} \Big/ \sum_i w_{y_i}.$$

The square root softens the re-weighting, and clipping prevents extreme weights.

Definition 5.2 (Cross-entropy plus soft Dice (segmentation)). With predicted probabilities p_k and one-hot targets y_k summed over the batch and pixels,

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \frac{1}{K}\sum_{k=1}^{K} \left(1 - \frac{2\sum p_k y_k + 1}{\sum p_k + \sum y_k + 1} \right).$$

Dice optimises region overlap directly and is far less dominated by the background class than cross-entropy alone, which matters when the target covers 2% of pixels.

5.3) Optimisation recipe

AdamW (weight decay 10^{-4}), cosine learning-rate annealing over the planned epochs, gradient clipping at norm 5, optional mixed precision, and early stopping on the validation metric with a configurable patience. Batch-norm running statistics of the pretrained encoder are frozen during fine-tuning because small final batches made them unstable. Every recipe is validated (allowed models, ranges for epochs, batch size, image size and learning rate) before any GPU time is spent.

5.4) Selection and inference

Definition 5.3 (Checkpoint selection). After every epoch the model is scored on the validation split; the checkpoint with the highest validation metric (accuracy for classification, mean IoU for segmentation) is saved. Across runs, the selected model is arg max of validation score; ties keep the earlier run.

Definition 5.4 (Flip test-time augmentation (TTA)). For the flip group $G = \{\mathrm{id}, h, v, hv\}$, predictions are averaged in probability space,

$$\hat{p}(x) = \frac{1}{4} \sum_{g \in G} g^{-1}(\operatorname{softmax} f(g(x))),$$

where g^{-1} un-flips segmentation maps. It is adopted only if it wins on validation, and it is recorded as a new, separate model version.

5.5) Metrics

Definition 5.5 (Dataset-wide IoU). Let C be the confusion matrix accumulated over *all* validation (or test) pixels. Then $\mathrm{IoU}_k = C_{kk} \,/\, (\sum_j C_{kj} + \sum_j C_{jk} - C_{kk})$, and mean IoU averages over classes with non-empty union (background and foreground for binary tasks).

Averaging per-batch IoUs would overweight small or empty images; the global confusion matrix avoids that. We also report foreground IoU, balanced accuracy, macro-F1 and per-class recall, and we never average accuracy with IoU across tasks.

6 The Agent: How GoTrain Decides

6.1) One tool call is one experiment

`run_experiment` takes a JSON recipe and a written *hypothesis*. The tool validates the recipe, caps its time by the remaining budget, trains, and returns the measured validation metrics. Failed runs consume an attempt, so the agent cannot retry forever, and every attempt (successes and failures) stays in the history.

6.2) What the system prompt enforces

- Inspect the data before training; prefer measured evidence over speculation; never invent scores or claim an unmeasured cause.
- Start from a pretrained ResNet-18 with known-good defaults (128 px and batch 32 for classification; 256 px, batch 8 and Dice for segmentation), then change a small number of settings at a time based on results.
- Keep the seed fixed so experiments are comparable; for a strong run that is still improving, prefer a warm start with a lower learning rate.
- Respect the budget (experiments and minutes); never request test data.
- Finish with scores, decisions, limitations and the selected run ID.

6.3) A real decision, quoted from the logs

Example 6.1. On `m-nz-cattle`, the agent's third hypothesis was: *"Cattle are small objects (∼2% pixels); higher input resolution (384) should improve foreground IoU."* It knew a warm start cannot change the image size, so it trained from pretrained weights at 384 px. Validation mean IoU rose from 0.8304 to 0.8445 and foreground IoU from 0.670 to 0.698. In its final summary it also wrote what it could *not* conclude: one seed, 66 validation images, and changes that were not isolated.

7 How We Achieved the Accuracy

Each table below lists every validation run in time order. Δ is the change against the best validation score so far. Runs R1 to R3 were chosen by the agent within a budget of three attempts (for PV4GER-seg the agent's attempts are R0 to R2, because its first attempt failed). The next run is a fixed, scripted refinement policy (fine-tune the selected checkpoint for up to 10 epochs at one third of its learning rate, at most 10^{-4}) and the last is the validation-only flip-TTA comparison of Definition 5.4. Neither of these two is presented as an agent decision.

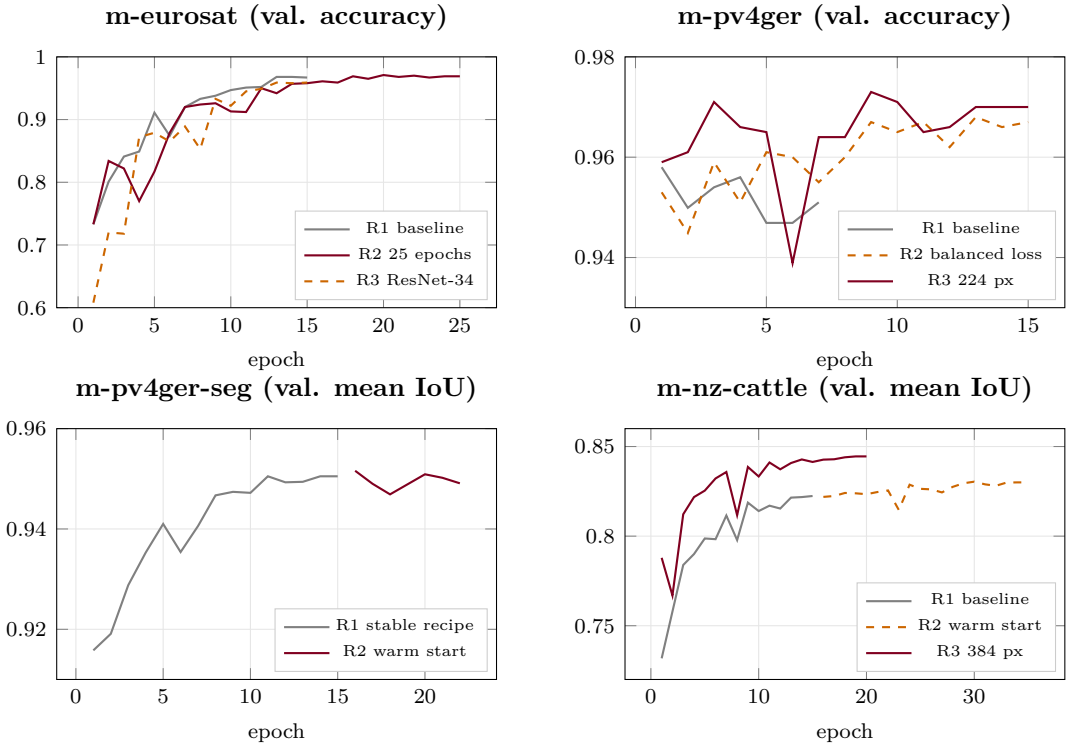


Figure 2: Per-epoch validation curves of the agent's runs, read from each run's `history.json`. Warm-start runs are drawn as a continuation of their parent's epochs.

7.1) EuroSAT: from 75% to 97%

Table 3: `m-eurosat` runs (validation accuracy).

Run	Hypothesis and change	Val. acc.	Δ
manual	3-layer CNN from scratch, 13 bands, Adam, 5 epochs (before GoTrain)	75.0%	–
R1	Pretrained ResNet-18, 128 px, 15 epochs (agent baseline)	96.8%	**+21.8**
R2	"Best epoch was 13/15, so the model may still be improving": 25 epochs	97.1%	**+0.3**
R3	Deeper ResNet-34 to separate Herbaceous Vegetation / Annual Crop	95.9%	−1.2 (rejected)
R4	Scripted refinement: fine-tune R2 at lr 10^{-4}	96.7%	−0.4 (rejected)
R5	Flip TTA on R2 (validation-only comparison)	**97.2%**	**+0.1**

The single largest gain of the whole project is **transfer learning with the 13-band adaptation**: +21.8 points over the from-scratch CNN. A bigger backbone was *worse* on 2,000 training images, and the agent kept the smaller model. The remaining errors are semantic: Herbaceous Vegetation is confused with Annual Crop (5 of 100 validation images) and vice versa (3 of 100). The same pair is the weak spot on test, where Annual Crop has the lowest recall (0.88) and Herbaceous Vegetation 0.94.

7.2) PV4GER classification: fixing missed panels

Table 4: `m-pv4ger` runs (validation accuracy; positive class is 15.6% of images).

Run	Hypothesis and change	Val. acc.	Δ
R1	Pretrained ResNet-18, 128 px. Best epoch was the first; positive recall 0.76	95.8%	–
R2	"Low positive recall, 36 false negatives": class-balanced loss	96.8%	**+1.0**
R3	"Panels are small details at 320 px native": input 224 px	97.3%	**+0.5**
R4	Scripted refinement of R3 at lr 10^{-4}	97.4%	**+0.1**
R5	Flip TTA on R4	**97.5%**	**+0.1**

Accuracy alone hid the real problem. The agent read the confusion matrix, saw that one in four panels was missed, and attacked recall twice: re-weighting the minority class, then giving the network more pixels per panel. Validation foreground IoU of the positive class rose from 0.736 to 0.841. On test, accuracy is 97.9% with balanced accuracy 0.934; the remaining weakness is still positive recall (0.869), which our diagnosis also reports (21 of 153 validation panels missed).

7.3) PV4GER segmentation: stability first

The first attempt diverged. The engine detected the non-finite loss, failed the run cleanly (the run is still recorded) and the agent diagnosed a plausible cause and fixed it on the next attempt. After that, the problem was largely solved by the first stable recipe; later steps added small, measured gains. Foreground IoU on test is 0.921.

Table 5: `m-pv4ger-seg` runs (validation mean IoU).

Run	Hypothesis and change	Val. mIoU	Δ
R0	U-Net, 256 px, lr 3×10^{-4}, mixed precision	failed	non-finite loss
R1	"Instability from AMP or high learning rate": lr 10^{-4}, AMP off	0.9505	–
R2	"Best epoch was the last": warm start at lr 5×10^{-5}	0.9516	**+0.0011**
R3	Scripted refinement at lr 1.7×10^{-5}	0.9519	**+0.0003**
R4	Flip TTA on R3	**0.9526**	**+0.0007**

7.4) NZ cattle: small objects need pixels

Table 6: `m-nz-cattle` runs (validation mean IoU; cattle are 2.2% of pixels).

Run	Hypothesis and change	Val. mIoU	Δ
R1	Pretrained U-Net, 256 px, CE + Dice	0.8224	–
R2	"Best epoch was the last": warm start, lr 10^{-4}, 20 epochs	0.8304	**+0.0080**
R3	"Cattle are ∼2% of pixels": 384 px input, trained from pretrained	0.8445	**+0.0141**
R4	Scripted refinement of R3 at lr 10^{-4}	0.8453	**+0.0008**
R5	Flip TTA on R4	**0.8476**	**+0.0023**

This is the hardest task: tiny objects, 524 training images and only 66 for validation. Resolution was the decisive lever. Our post-hoc diagnosis of the final model counts 2,559 annotated animals in validation and finds 99% of the small ones and 96% of the large ones at the object level, with a predicted area 1.05× the true area, so the remaining IoU gap is mostly about exact boundaries of rasterised annotations rather than missed animals.

7.5) What worked across tasks

Table 7: Measured effect of each lever (validation, our runs).

Lever	Evidence	Verdict
ImageNet pretraining + band adaptation	EuroSAT 75.0% → 96.8%	essential
Higher input resolution	PV4GER +0.5 pt; cattle +0.0141 mIoU	helps small objects
Class-balanced loss	PV4GER +1.0 pt, positive recall up	helps rare classes
Training longer / warm start	EuroSAT +0.3 pt; cattle +0.0080; seg +0.0011	helps if still improving
Lower-LR refinement	+0.0003 to +0.001 on 3 tasks, −0.4 pt on EuroSAT	small, not always
Flip TTA	better on **4 of 4** tasks (+0.0007 to +0.0023)	cheap, consistent
Deeper backbone (ResNet-34)	EuroSAT −1.2 pt	rejected

All four final models used about 64 GPU-minutes of training on one T4 in total. Speed came from

pretrained backbones, cached inputs, early stopping and a hard experiment budget.

8 Final Test Results

Table 8: Full test metrics of the four frozen models.

Task	Accuracy	Balanced acc.	Macro-F1	Mean IoU	Foreground IoU
`m-eurosat`	**0.9570**	0.9570	0.9569	0.9189	–
`m-pv4ger`	**0.9790**	0.9340	0.9573	0.9197	0.8636
`m-pv4ger-seg`	0.9853	0.9749	0.9749	**0.9514**	0.9206
`m-nz-cattle`	0.9913	0.9202	0.9084	**0.8439**	0.6967

Validation versus test. The gaps are −1.5 points (EuroSAT), +0.4 points (PV4GER), −0.0012 (PV4GER-seg) and −0.0037 (cattle). Small gaps on three tasks indicate that selecting on validation did not overfit; EuroSAT's larger drop is consistent with many sequential choices made on one 1,000-image validation set.

Traceability. Every score above links to a selected run ID, its checkpoint SHA-256, the evaluation time and the original `test-result.json`, and the metrics were re-verified from the saved confusion matrices (Figure 3).

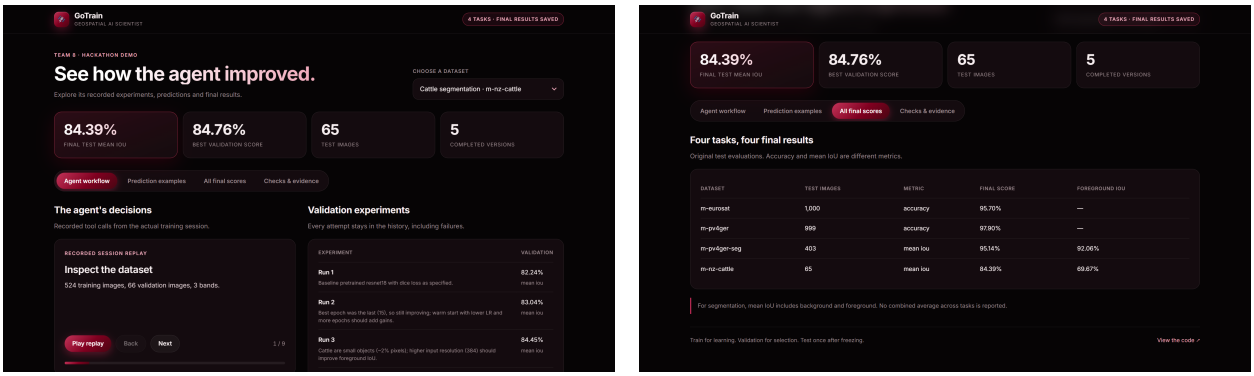


Figure 3: Results dashboard built from the exported evidence: per-task metrics and agent replay (left), all four final scores (right).

9 What Makes GoTrain Innovative

The scored runs above already used hypothesis-driven experiments. On top of them we built the features that turn GoTrain from a tuning loop into an **AI scientist**. They were built after the scored runs and were *not* used to produce the final scores; they were run on the frozen models using validation data only.

9.1) Error-driven diagnosis

Definition 9.1 (Object-level detection by size). Each true foreground mask is split into 8-connected components (one component per object). An object is *detected* if at least 50% of its pixels are predicted as foreground. Objects are grouped into small / medium / large by the terciles of their areas, and the detection rate is reported per group.

`diagnose_errors` also reports predicted-to-true foreground area, false positives on images with no target, the most-confused class pairs, the weakest class recall, and the worst validation images as pictures. It then lists *candidate levers*, for example "small objects detected less often ⇒ try `focus_crops` or a larger image size". The agent must quote one of these measured numbers in its next hypothesis.



Figure 4: A worst validation image found by `diagnose_errors` on PV4GER-seg: input, true mask, prediction. Its diagnosis: small panels found 88% of the time versus 99% for larger ones.

9.2) Targeted fixes and verdicts

`focus_crops` is a new segmentation option: half of the training samples are cropped to a half-size window around a random foreground pixel (with jitter) and resized back, so objects appear twice as large; image and mask stay aligned. After every experiment the tool returns a verdict, `kept` only if the run strictly beats the previous best validation score, together with the minutes and attempts left.

9.3) Memory across datasets

Definition 9.2 (Lesson). A lesson is the recipe difference between a run and the best run it had to beat, together with the measured validation change and a verdict (*kept*, *rejected* or *failed*). Lessons are derived mechanically from run records, so they cannot be invented. The agent can add *insights*, but each must cite run IDs from the task.

Before its first experiment on a new dataset, the agent calls `recall_lessons` and receives lessons from other tasks (same problem type first), a scoreboard per lever and the winning recipe of each similar task. Lessons are hints to re-test, never results. Reconstructed from our real runs, the memory holds 15 lessons across the four tasks; for instance flip TTA was kept 4 of 4 times, and $256 \rightarrow 384$ px gave +0.0142 mIoU on cattle.

9.4) A product, not a notebook



Figure 5: Before and after on the first model's hardest validation images (input, first model, selected model, true mask). Cattle: foreground IoU 0.51 → 0.56. Solar panels: 0.30 → 0.45. These images were chosen because they were the worst, not because they improved; one of the three PV4GER-seg examples stayed at 0.

- **Chat interface.** Type "find cattle in aerial photos, 45 minutes, 3 experiments"; GoTrain parses the task, budget and number of experiments (and asks when unclear), starts the agent and streams every step as a chat message (Figure 6).
- **Model registry.** All 20 model versions with lineage, checksums and badges (best, published, verified, frozen); one-click publish and restore-and-verify; an audited storage log. In our workspace, 11 versions are published and 4 were restored with every checksum matched.
- **Demo video and dashboard** built from the recorded evidence.
- **GoPilot-ready.** `python -m gotrain.mcp_server` exposes inspection, training, diagnosis, memory, comparison and model upload/download as MCP tools, with no test-split access.

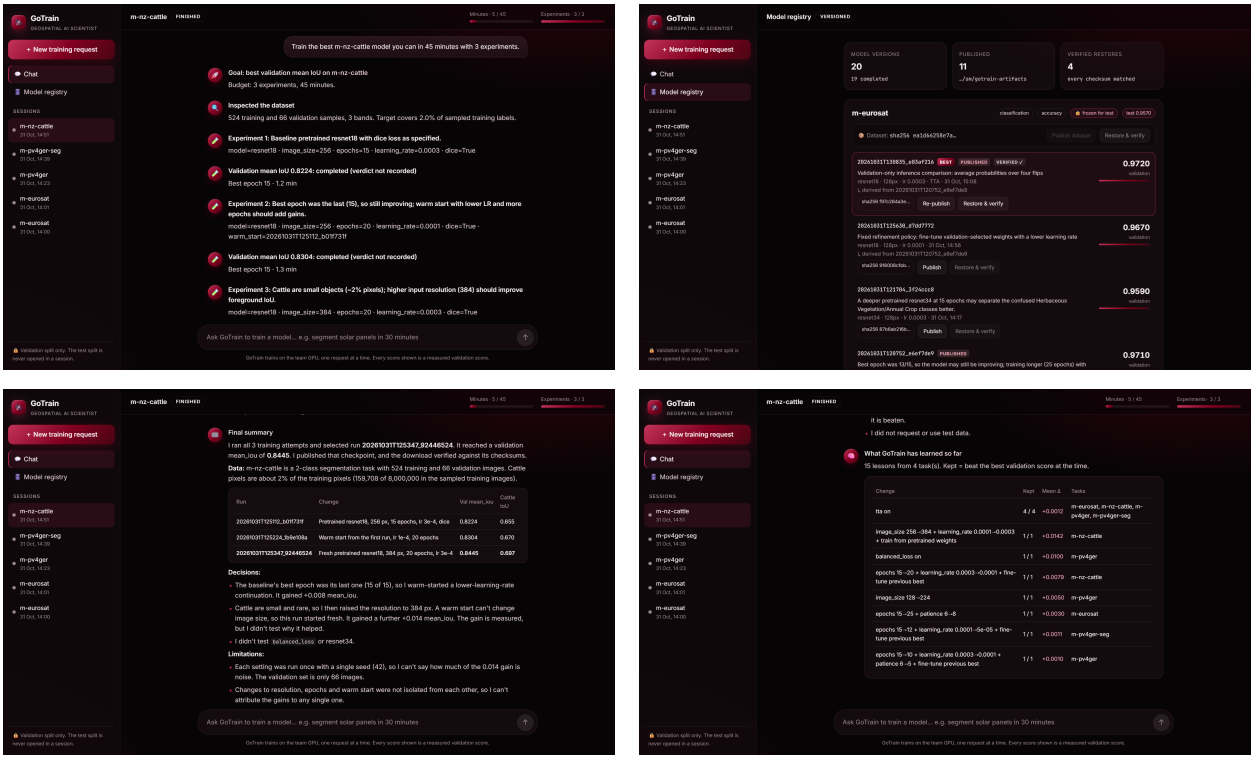


Figure 6: The GoTrain chat app: a real agent session (top left), the model registry (top right), the agent's own final summary (bottom left) and the cross-task memory scoreboard (bottom right).

10 Evaluation Integrity and Engineering

10.1) Test exactly once

`gotrain finalize TASK -confirm-test-once` (1) selects the best validation run, (2) writes `final-selection.json` with an exclusive create, (3) verifies the checkpoint hash and the dataset fingerprint, (4) writes `test-started.json`, again with an exclusive create, *before* the first test sample is read, and (5) evaluates. A second call fails before touching the test split, a crash after step 4 stays locked, and training on a frozen task is refused. A file lock also prevents two GPU jobs from running at the same time.

10.2) Reproducibility and provenance

Each run stores its recipe, seed, hypothesis, git commit, package versions, GPU name, dataset fingerprint (hashes of the task files), normalisation sample indices and the full per-epoch history. Checkpoints are written atomically and hashed; warm starts and inference variants record their parent's hash, so model lineage is verifiable.

10.3) Testing

34 automated tests run on synthetic fixtures in CI: metrics, the train/valid-only guard, train-only normalisation, mask alignment, focus crops, global IoU, checkpoint corruption, once-only finalisation, warm-start lineage, TTA un-flipping, diagnosis and lessons, the agent's budget and logs with a mocked model, the chat server, the registry, and the dashboard's escaping of untrusted text.

11 Honest Limitations

Remark 11.1. We prefer to state these ourselves; they are also written in our logs and README.

- **Single seed.** Each setting ran once (seed 42); differences of $\sim$0.001 are within noise, especially on 66 cattle validation images.
- **Confounded changes.** Some runs changed several settings together (for example resolution, learning rate and initialisation for cattle R3), so a gain cannot be attributed to one setting alone.
- **Scripted steps.** The refinement (R4) and TTA (R5) steps were fixed policies, not agent decisions; we report them separately.
- **New features after scoring.** Diagnosis, focus crops, verdicts and memory did not produce the final scores; memory was reconstructed from the runs after freezing.
- **Comparison protocol.** We did not reproduce the GEO-Bench paper's exact protocol (budget, seeds, pretraining), so we do not claim a ranking against it.
- **Deployment.** Storage was verified on the filesystem; cloud object storage and a live GoPilot

deployment were not exercised.

12 What We Would Build Next at RASID

1. **Ship GoTrain as an MCP server inside GoPilot**, with authentication and cloud object storage for datasets and model versions.
2. **Close the scientist loop on new data**: run diagnosis → targeted fix (`focus_crops`, resolution, balancing) → verdict end to end, and measure how much recalled memory shortens the search on a fresh dataset.
3. **Statistical rigour**: multiple seeds and confidence intervals before a change is *kept*; isolate one change per experiment.
4. **Stronger backbones**: geospatial foundation models pretrained on multispectral imagery, compared under the same protocol.
5. **Search that branches**: explore several promising recipes in parallel (tree search) when more than one GPU is available.

In one sentence: GoTrain trains strong geospatial models on its own, explains every decision with a measured hypothesis, learns from one dataset to the next, and proves each final score with an auditable, test-once trail.

13 Appendix: Reproduce and Verify

13.1) Commands

```
python -m pip install -e '.[dev]'
python -m pytest -q                                    # 34 tests
python -m gotrain prepare m-nz-cattle
python -m gotrain agent m-nz-cattle --experiments 3 --minutes 45
python -m gotrain diagnose m-nz-cattle                 # validation only
python -m gotrain learn m-nz-cattle                    # lessons from runs
python -m gotrain ui                                   # chat + registry
python -m gotrain finalize m-nz-cattle --confirm-test-once
python -m gotrain export --out submission
```

13.2) Frozen model versions

Code: `https://github.com/mastercoder67/GoTrain-Hackathon`.

Task	Selected run (validation-chosen, then tested once)
`m-eurosat`	`20261031T130835_e03af216` (R2 + flip TTA)
`m-pv4ger`	`20261031T130848_2e474ff8` (R4 + flip TTA)
`m-pv4ger-seg`	`20261031T130907_dd275c19` (R3 + flip TTA)
`m-nz-cattle`	`20261031T130927_b6c58559` (R4 + flip TTA)